

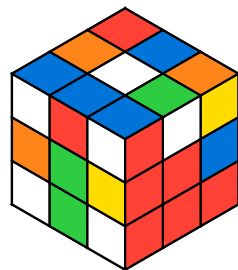
The "magic-cubes" package

v0.0.1 MIT

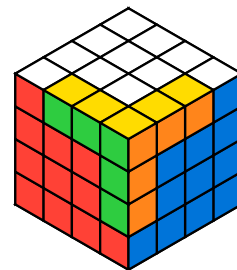
Represent Rubik's cubes of arbitrary size and apply algorithms to them

RODRIGO ALCÁNTARA

 @RodAlc24



U' L' U' F' R2 B'
R F U B2 U B' L
U' F U R F'



F U2 L F L' B L
U B' R' L' U R' D'
F' B R2

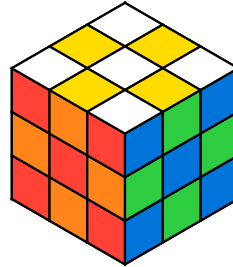
Table of Contents

Examples	2	III.2 Central moves	11
Guide	4	III.3 Wide moves	11
II.1 Creating cubes	4	III.4 Cube rotations	12
II.2 Applying algorithms	6	III.5 Modifiers	13
II.3 Rendering cubes	7	Reference	14
II.3.1 Flat representation	7	IV.1 Custom types	14
II.3.2 3D cube	7	IV.2 Functions	15
II.3.3 Face view	8	IV.2.1 Cubes	15
II.3.4 Other views	9	IV.2.2 Algorithms	16
II.3.5 OLL view	9	IV.2.3 Render	18
II.3.6 PLL view	9	Index	22
Cube notation	10		
III.1 1-layer moves	10		

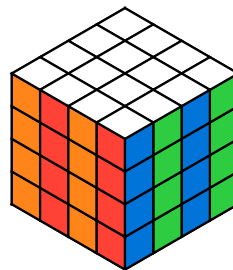
Part I

Examples

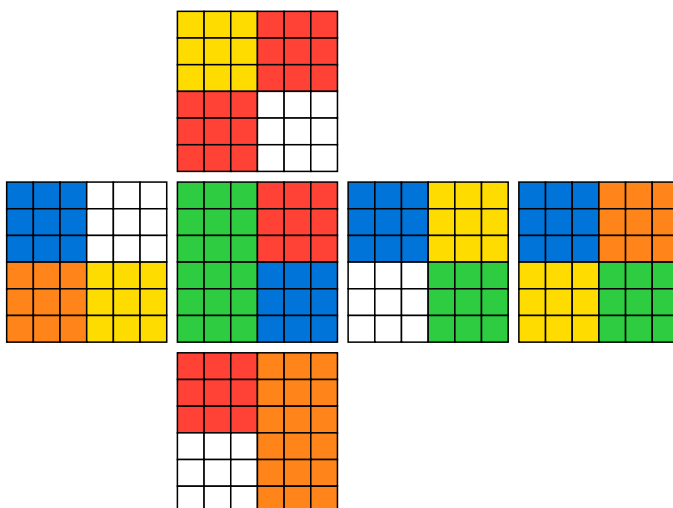
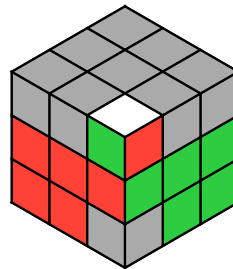
```
#draw_cube(
  apply(
    cube(),
    "F2 B2 R2 L2 U2 D2"
  )
)
```



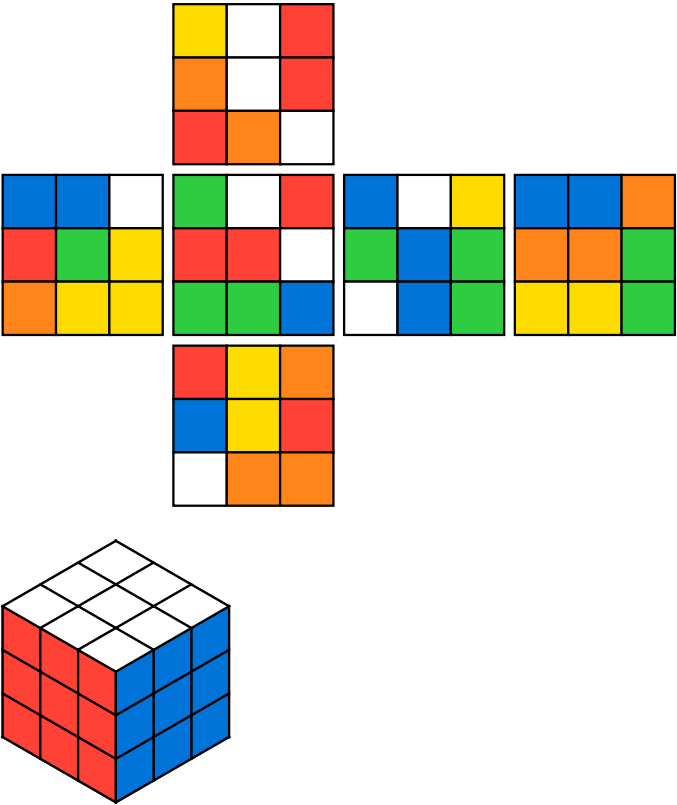
```
#draw_cube(
  apply(
    cube(size: 4),
    "R2 3R2 F2 3F2 R2 3R2"
  )
)
```



```
#draw_cube(
  apply(
    inverted: true,
    f2l-cube,
    "R2 U R2 U R2 U2 R2"
  )
)
```



I Examples



Part II

Guide

II.1 Creating cubes

A cube is represented as a dictionary, and can be easily created with the `#cube` function. It should not be created or modified manually.

The size of the cube can be specified with the `(size)` argument (default is 3). The face colors can be changed with the `(colors)` argument, specifying the faces to change. The default colors are:

```
(  
  f: red,  
  r: blue,  
  u: white,  
  b: orange,  
  l: green,  
  d: yellow  
)
```

This colors are the Typst [predefined colors](#)¹.

It is also possible to specify a face manually, in that case the corresponding value in `(colors)` is ignored. When specifying the colors manually, all the stickers in the face must be determined.

The order of the stickers in a face is the following:

			0	1	2			
			3	4	5			
			6	7	8			
0	1	2	0	1	2	0	1	2
3	4	5	3	4	5	3	4	5
6	7	8	6	7	8	6	7	8
			0	1	2			
			3	4	5			
			6	7	8			

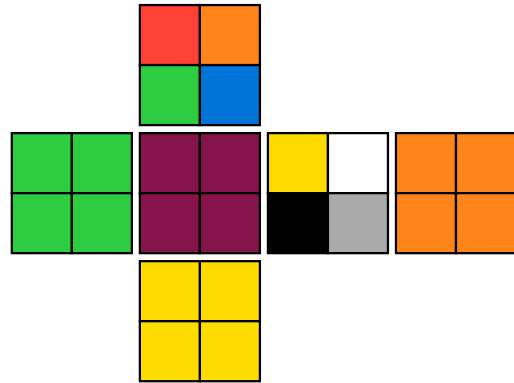
Each face is represented with the default color previously explained, except the up face that is in black.

¹<https://typst.app/docs/reference/visualize/color/#predefined-colors>

Note that this way of creating cubes may result in an impossible cube state (you can even use more than six colors!). This can be useful, for example, to add a gray color to the non relevant pieces.

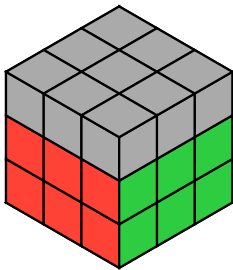
A similar order is used in other size cubes, for example, if we want to create a 2x2x2 cube with a custom color in front face and custom up and right faces we can do:

```
#draw_flat(
  length: 45pt,
  cube(
    size: 2,
    colors: (f: maroon),
    stickers: (
      u: (
        red,
        orange,
        green,
        blue,
      ),
      r: (
        yellow,
        white,
        black,
        gray,
      ),
    ),
  )
)
```

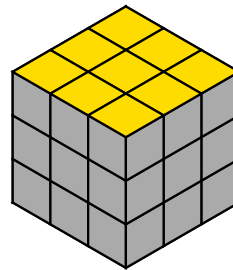


There are also some useful predefined cubes: `#f2l-cube` and `#oll-cube`.

```
#draw_cube(f2l-cube)
```



```
#draw_cube(oll-cube)
```

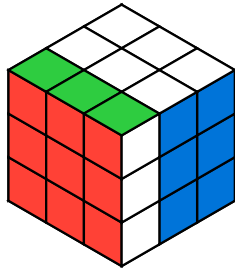


II.2 Applying algorithms

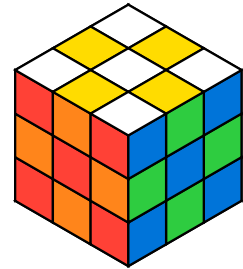
One of the main features of **MAGIC-CUBES** is the possibility to apply algorithms to the cubes. Algorithms are written following the standard notation that is fully documented on [Section III](#).

The function `#apply` receives a cube and an algorithm as a `str`. It is also possible, through the `(inverted)` argument to apply the inverse algorithm. This results in a cube that, after applying the specified algorithm, will result in the original cube.

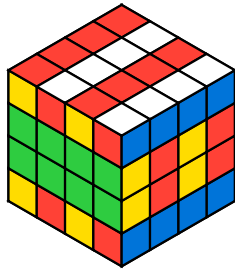
```
#draw_cube(
  apply(
    cube(),
    "F"
  )
)
```



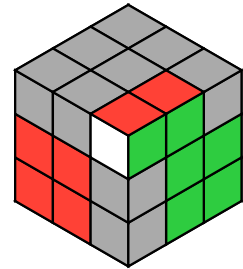
```
#draw_cube(
  apply(
    cube(),
    "M2 E2 S2"
  )
)
```



```
#draw_cube(
  apply(
    cube(size: 4),
    "2R L' 2-3u'"
  )
)
```

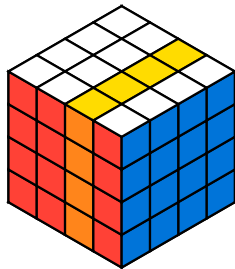


```
#draw_cube(
  apply(
    f2l-cube,
    inverted: true,
    "U R U' R'"
  )
)
```

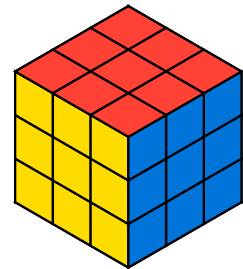


There are also two alternative functions that lets you modify the cube: `#rotate_layer` and `#rotate_cube`.

```
#draw_cube(
  rotate_layer(
    cube(size: 4),
    "r",
    depth: 2,
    n: 2
  )
)
```



```
#draw_cube(
  rotate_cube(
    cube(),
    "x"
  )
)
```

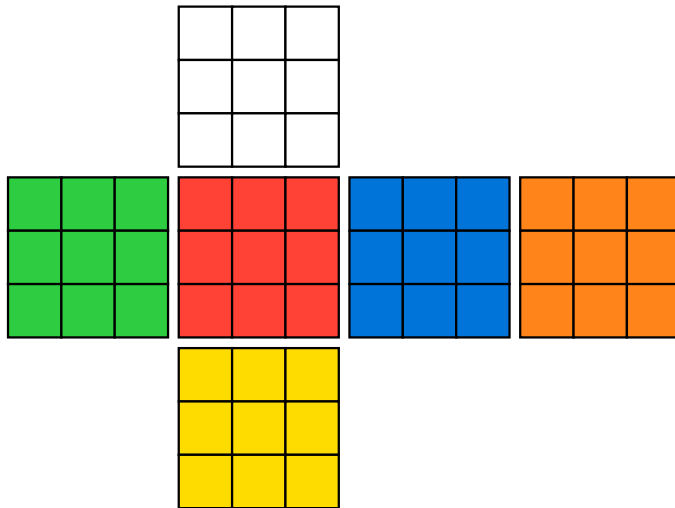


II.3 Rendering cubes

II.3.1 Flat representation

It is possible to get a full representation of the cube with `#draw_flat`. This function takes a `cube` and draws its net. It also accepts a `(length)` argument to change the length of each cube edge.

```
#draw_flat(cube())
```



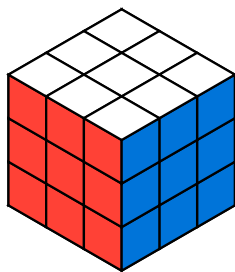
The faces are displayed in the standard cube net layout. The center row contains the left, front, right and back faces. The upper face is placed above the front face, and the down face is placed below it.

II.3.2 3D cube

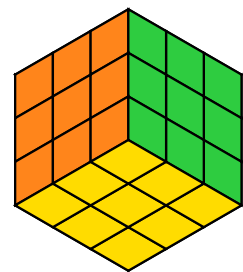
Another option is to draw a three-dimensional representation of the cube. This is done with `#draw_cube`.

Apart from the `(cube)` and `(length)` arguments, which behave the same as in `#draw_flat`, it also accepts `(x)`, `(y)` and `(z)` arguments to customize the cube orientation. By default, the cube is drawn in an isometric projection.

```
#draw_cube(  
  cube(),  
)
```



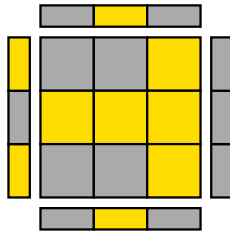
```
#draw_cube(  
  cube(),  
  x: -35.264deg,  
  y: 225deg,  
)
```



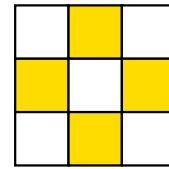
II.3.3 Face view

The third type of representation is achieved with `#draw_face`. This draws a single face, optionally including the first row of the adjacent faces. This view is commonly used to illustrate Orientation of the Last Layer (OLL) and Permutation of the Last Layer (PLL) algorithms.

```
#draw_face(
  apply(
    oll-cube,
    "F R U R' U' F'",
    inverted: true
  ),
  "u"
)
```

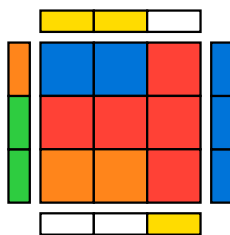


```
#draw_face(
  apply(
    cube(),
    "M2 E2 S2"
  ),
  "u",
  lateral-faces:
  false
)
```

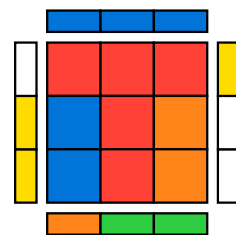


In addition to the face to display, you may also specify the face that will be displayed on the top to control the orientation of the cube. This argument defaults to `auto`, which means that the `(up-face)` will take the value of "u" if `(face)` is set to "f", "r", "b" or "l"; "f" if `(face)` is "d" and "b" if `(face)` is "u".

```
#draw_face(
  apply(
    cube(),
    "F R l U' R2",
    inverted: true
  ),
  "u"
)
```



```
#draw_face(
  apply(
    cube(),
    "F R l U' R2",
    inverted: true
  ),
  "u",
  up-face: "r"
)
```



II.3.4 Other views

There are also 3 other functions for rendering cubes. These are wrappers of the previous ones.

#draw_f2l

This function displays the cube (by default #f2l-cube) in a useful way for documenting F2L algorithms. #draw_f2l requires an algorithm that will be applied to the cube and displayed below it.

```
#draw_f2l(
  "(R U' R' U) (R
  U' R')",
  length: 45pt
)
```



(R U' R' U)
(R U' R')

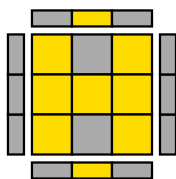
```
#draw_f2l(
  "d (R' U2 R) d' (R
  U R')",
  length: 45pt
)
```



d (R' U2 R) d'
(R U R')

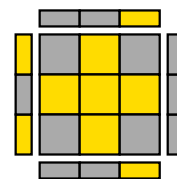
II.3.5 OLL view

```
#draw_oll(
  "(R U R' U') r R'
  (U R U' r')",
  length: 45pt
)
```



(R U R' U') r R'
(U R U' r')

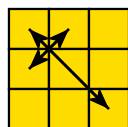
```
#draw_oll(
  "R U2 R2 U' R2 U'
  R2 U2 R",
  length: 45pt
)
```



R U2 R2 U' R2 U'
R2 U2 R

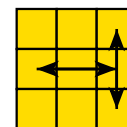
II.3.6 PLL view

```
#draw_pll(
  "F R U' R' U' R U
  R' F' R U R' U' R' F
  R F'",
  length: 45pt
)
```



F R U' R' U' R U
R' F' R U R' U'
R' F R F'

```
#draw_pll(
  "R U R' U' R' F R2
  U' R' U' R U R' F'",
  length: 45pt
)
```



R U R' U' R' F
R2 U' R' U' R U
R' F'

Part III

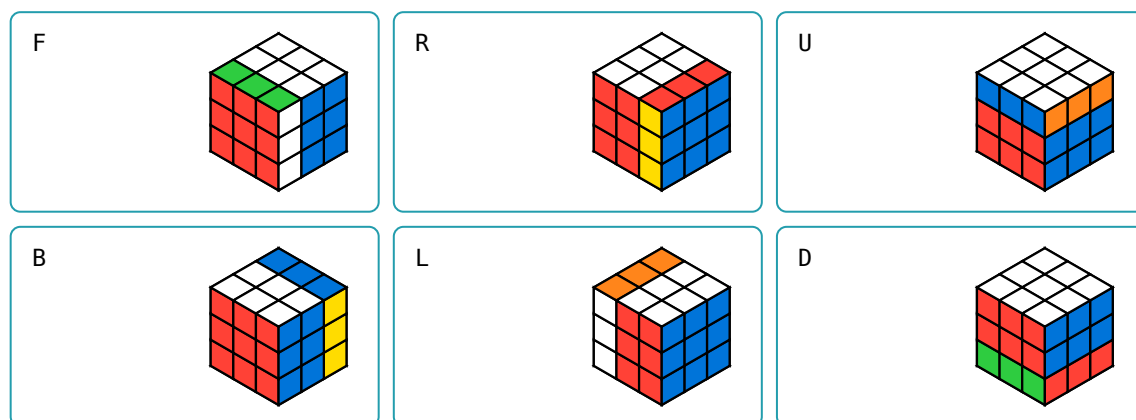
Cube notation

Algorithms are written as a string, consisting of a sequence of moves separated by spaces. These moves may represent rotations of one or more layers, or even of the entire cube.

Parenthesis may also be used for clarification, they are ignored by the parser. Any other character that is not part of the valid notation will cause an error.

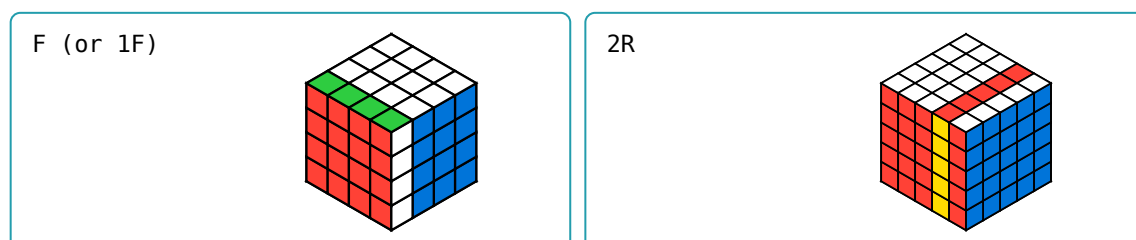
III.1 1-layer moves

The basic moves are represented with one uppercase letter. There are six moves, one for each face of the cube: **F** (front), **R** (right), **U** (upper), **B** (back), **L** (left) and **D** (down). Each represents a single clockwise rotation. Double and counterclockwise rotations are explained in [Section III.5](#).

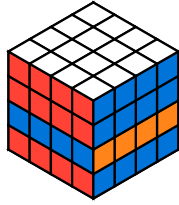


It is also possible to move an inner layer. To do so, precede the letter with the depth of the layer. The outermost layer has a depth of 1 and layer numbering always starts from the referenced face.

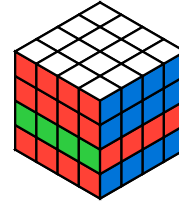
The maximum depth is the cube size minus 1. If you want to rotate the opposite face use the corresponding notation, i.e., in a 4x4x4 cube a 4F rotation is not allowed, the correct notation is B'.



3U



2D



It is not necessary to specify the depth for the moves of the outermost layer. For example, 1U is equivalent to U in all cubes.

III.2 Central moves

The central layers have a special notation:

- **M**: “middle”, between L and R following L.
- **E**: “equator”, between U and D following D.
- **S**: “standing”, between F and B following F.

M



E



S



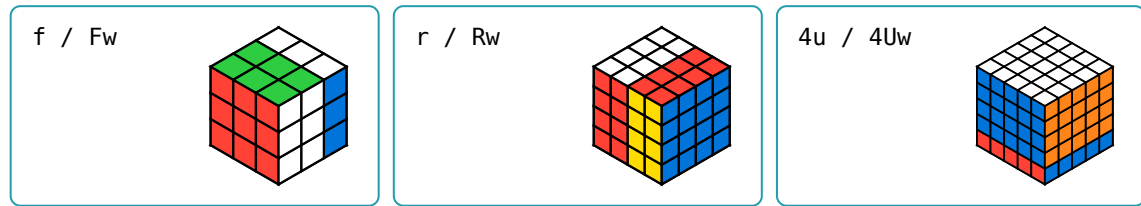
Central moves are only available on cubes with odd size.

III.3 Wide moves

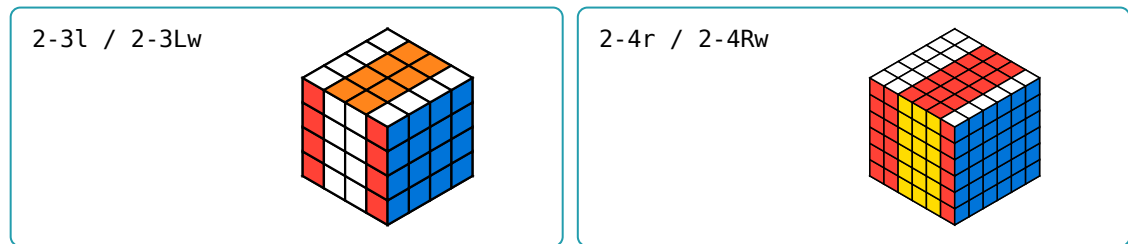
It is also possible to rotate more than one layer at the same time. This is especially useful for big cubes, but it is also used sometimes on a 3x3x3.

There are two equivalent notations: using lowercase letters or appending a **w** after the corresponding uppercase letter. By default, all layers between the outermost layer and the center (if the cube has odd size) are rotated, but this can be specified with a number before the move. For example, **3f** (or **3Fw**) rotates the first three front layers.

Just as before with the depth parameter, the maximum number of layers that can be moved with a wide move is equal to size - 1. A specific notations exists for rotating the entire cube that will be explained next.



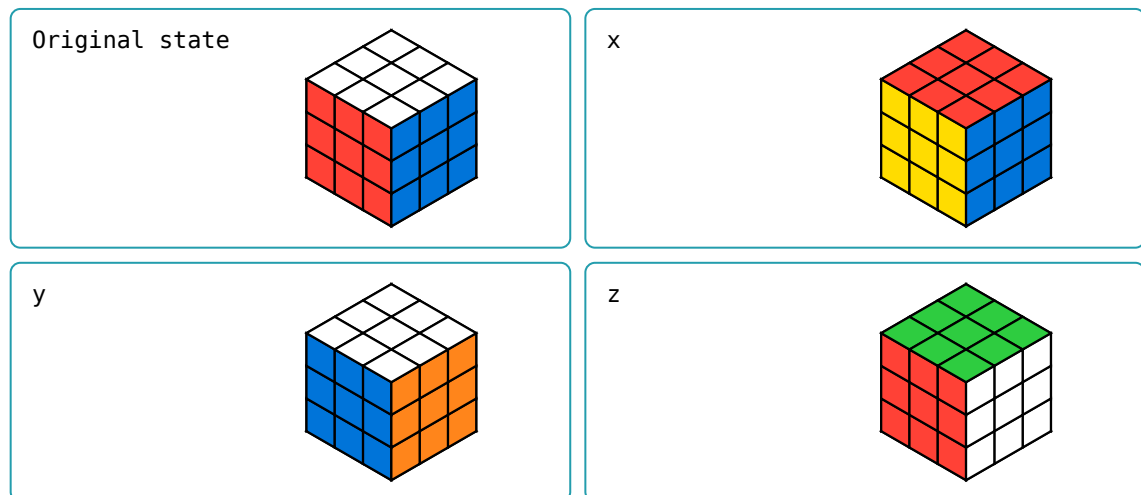
It is also possible to rotate multiple inner layers. To do so, write the first and last layers before the move separated with a dash.



III.4 Cube rotations

For a complete cube rotation the letters **x**, **y** and **z** are used. These movements do not alter the state of the cube, just the point of view.

- **x**: rotating the whole cube as if performing **R**.
- **y**: rotating the whole cube as if performing **U**.
- **z**: rotating the whole cube as if performing **F**.



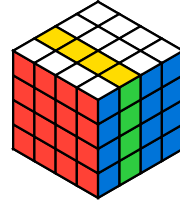
III.5 Modifiers

Appending an apostrophe (') or a “2” to a move denotes an counterclockwise rotation or a double one (180°), respectively. These modifiers can be applied to any notation described above.

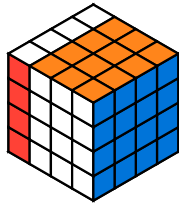
R'



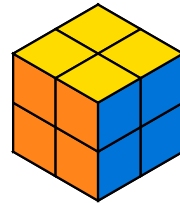
2F2



3Rw'



x2



Part IV

Reference

IV.1 Custom types

IV.1.1 `face`

`face` is the type used throughout the package to refer to one of the six faces of the cube. It accepts the following `str` values:

- "f": for the **f**ront face.
- "r": for the **r**ight face.
- "u": for the **u**pper face.
- "b": for the **b**ack face.
- "l": for the **l**eft face.
- "d": for the **d**own face.

Face identifiers are lowercase and case-sensitive.

IV.1.2 `cube`

`cube` represents the complete state of a Rubik's cube, it is the main element in this package. However, you should not create or modify instances manually, as functions such as `#draw_cube` require the cube to be in a valid, consistent state. Instead, use `#cube` to create instances and `#apply` to manipulate them.

```
(  
  {size} number  
  {f} array of color  
  {r} array of color  
  {u} array of color  
  {b} array of color  
  {l} array of color  
  {d} array of color  
)
```

Each array must contain `{size}`² elements.

IV.1.3 cube-colors

This type is used when creating a cube to specify the color of each face.

```
(
  {f} color
  {r} color
  {u} color
  {b} color
  {l} color
  {d} color
)
```

Not all keys need to be present for a valid `cube-colors` value.

IV.1.4 cube-stickers

This type is used when creating a cube to specify the color of the stickers for each face. All the arrays must have the same length and it must be equal to the square of the size of the cube.

```
(
  {f} array of color
  {r} array of color
  {u} array of color
  {b} array of color
  {l} array of color
  {d} array of color
)
```

Not all keys need to be present for a valid `cube-stickers` value.

IV.2 Functions**IV.2.1 Cubes**

```
#cube
```

#cube(({size}: 3, {colors}: auto, {stickers}: auto) → **cube**

This function creates a **cube** given the size, colors, and/or stickers. This is the correct way to create **cube** instances.

The `<stickers>` argument lets you specify all the stickers on a face. Note that if a face is specified with this argument, it must be fully specified, and the corresponding entry in `<colors>` is ignored.

For each face present in `<stickers>`, the corresponding value in `<colors>` is ignored.

Argument —
`<size>`: 3 int
 The size of the cube. The default value is 3, i.e., a standard cube.

Argument —
`<colors>`: auto cube-colors | auto
 The colors assigned to each face.
 It must be a `cube-colors`, although it does not need to contain all the keys. In that case, the default color value will be used.
 The default values for each face are:

```
(
  f: red,
  r: blue,
  u: white,
  b: orange,
  l: green,
  d: yellow
)
```

Argument —
`<stickers>`: auto cube-stickers | auto
 The stickers of the cube. By default, the cube is initialized in the solved state. See [Section II.1](#) for more details and examples.

`#f2l-cube` cube

3x3x3 cube with only the first 2 layers colored. The upper layer is in gray color.

`#oll-cube` cube

3x3x3 cube with only the upper face colored. The rest of the cube is in gray color.

`#solved-cube` cube

3x3x3 solved cube.

IV.2.2 Algorithms

`#apply` `#rotate_cube` `#rotate_layer`

#apply(**{cube}**, **{alg}**, **{inverted}**: **false**) → **cube**

Returns a new **cube** with an algorithm applied.

Argument

{cube}

cube

The cube to which the algorithm is applied.

Argument

{alg}

str

A string containing the algorithm to apply, following the notation specified in [Section III](#).

Argument

{inverted}: **false**

bool

If true, the inverse of the algorithm is applied. This is useful when documenting solution algorithms, which are intended to solve the resulting cube.

#rotate_cube(**{cube}**, **{axis}**) → **cube**

Returns a new **cube** rotated.

It returns the cube after applying the rotation. It is used for applying the rotations explained in [Section III.4](#).

Argument

{cube}

cube

The cube to apply the rotation.

Argument

{axis}

str

The axis around which the cube is rotated. Must be one of ("x", "y", "z").

#rotate_layer(**{cube}**, **{face}**, **{depth}**: **1**, **{n}**: **1**) → **cube**

Returns a new **cube** with one layer rotated.

It returns the cube after applying the rotation. It is used for applying the 1-layer rotations explained in [Section III.1](#).

Argument

{cube}

cube

The cube to apply the rotation.

Argument

{face}

face

The face that defines the layer to rotate, must be one of ("f", "r", "u", "b", "l", "d").

Argument

(depth): 1

int

The depth of the layer to rotate, by default 1, i.e., the outermost face.
It must be smaller than the cube size.

Argument

(n): 1

int

The number of rotations to apply, by default 1.

IV.2.3 Render

#draw_cube
#draw_f2l

#draw_face
#draw_flat

#draw_oll
#draw_pll

```
#draw_cube(  
  {cube},  
  {x}: 35.264deg,  
  {y}: 45deg,  
  {z}: 0deg,  
  {length}: 60pt  
) → content
```

Draws a 3D representation of the cube.

The orientation can be modified with the {x}, {y} and {z} arguments. The default orientation is an isometric projection. For more details, see the [CETZ documentation²](https://cetz-package.github.io/docs/api/draw-functions/projections/ortho).

Argument

{cube}

cube

The cube to draw.

Argument

{x}: 35.264deg

angle

Rotation around the x-axis.

Argument

{y}: 45deg

angle

Rotation around the y-axis.

²<https://cetz-package.github.io/docs/api/draw-functions/projections/ortho>

Argument —
`{z}: 0deg` angle
 Rotation around the z-axis.

Argument —
`{length}: 60pt` length
 The length of the side of the cube.
 Note that it is different from the cube length. `{length}` specifies the length of **one cube edge before projection**.

#draw_f2l(`{alg}`, `{cube}: f2l-cube`, `{length}: 60pt`)

Draws a cube after applying the algorithm alongside with the algorithm. Default cube is `#f2l-cube`.

Argument —
`{alg}` str
 The algorithm to apply.

Argument —
`{cube}: f2l-cube` cube
 The cube to draw.

Argument —
`{length}: 60pt` length
 The length of the side of the cube. See `#draw_cube`.

#draw_face(
`{cube}`,
`{face}`,
`{up-face}: auto`,
`{length}: 60pt`,
`{lateral-faces}: true`
`)` → content

Draws a single face of the cube, and optionally the first row of the adjacent faces.

Argument —
`{cube}` cube
 The cube to draw.

Argument —
`{face}` face
 The face to display.

It must be one of ("f", "r", "u", "b", "l", "d").

Argument

(up-face): **auto**

face | **auto**

The face that will be displayed above the main face. It defines the orientation of the cube. If set to **auto**, the logical default is used:

- For "f", "r", "b" and "l": "u".
- For "u": "b".
- For "d": "f".

Valid values are **auto** and any **face** except the selected in (face) and its opposite.

Argument

(length): **60pt**

length

The length of the side of the cube.

Argument

(lateral-faces): **true**

bool

Whether lateral faces will appear or not.

#draw_flat((cube), (length): **60pt**) → **content**

Draws a flat representation of the cube.

Argument

(cube)

cube

The cube to draw.

Argument

(length): **60pt**

length

The length of the side of the cube.

#draw_oll((alg), (cube): **oll-cube**, (length): **60pt**)

Draws a face after applying the algorithm alongside with the algorithm. Default cube is **#oll-cube**.

Argument

(alg)

str

The algorithm to apply.

Argument

(cube): **oll-cube**

cube

The cube to draw.

Argument

`{length}: 60pt`

length

The length of the side of the cube. See `#draw_cube`.

`#draw_pll(``{alg},``{cube}: solved-cube,``{lateral-faces}: false,``{arrows}: true,``{length}: 60pt``)`

Draws a face after applying the algorithm alongside with the algorithm. It may also show arrows with the movement of the pieces on the upper face. Default cube is `#oll-cube`.

Argument

`{alg}`

str

The algorithm to apply.

Argument

`{cube}: solved-cube`

cube

The cube to draw.

Argument

`{lateral-faces}: false`

bool

Whether to show or not the lateral faces. See `#draw_face`

Argument

`{arrows}: true`

bool

Whether to show or not arrows representing the moves.

Argument

`{length}: 60pt`

length

The length of the side of the cube. See `#draw_cube`.

Part V

Index

A

`#apply` 6, 14, 16, 17

C

`#cube` 4, 14, 15

`cube` 14

`cube-colors` 15

`cube-stickers` 15

D

`#draw_cube` ... 7, 14, 18, 19, 21

`#draw_f2l` 9, 18, 19

`#draw_face` ... 8, 18, 19, 21

`#draw_flat` 7, 18, 20

`#draw_oll` 18, 20

`#draw_pll` 18, 21

F

`#f2l-cube` 5, 9, 16, 19

`face` 14

O

`#oll-cube` 5, 16, 20, 21

R

`#rotate_cube` 6, 16, 17

`#rotate_layer` 6, 16, 17

S

`#solved-cube` 16